

Segurança Computacional

Sumário

1	Segurança de Redes	3
1.1	O que é segurança de rede?	3
1.2	Princípios da criptografia	4
1.3	Uso da Criptografia de Chave Pública	8
1.3.1	Autenticação	8
1.3.2	Assinaturas Digitais e Integridade de Mensagens	11
1.3.3	Acordo de Chave Compartilhada	14
1.4	E-mail seguro	14
1.5	Proteção de conexões TCP: TLS	15

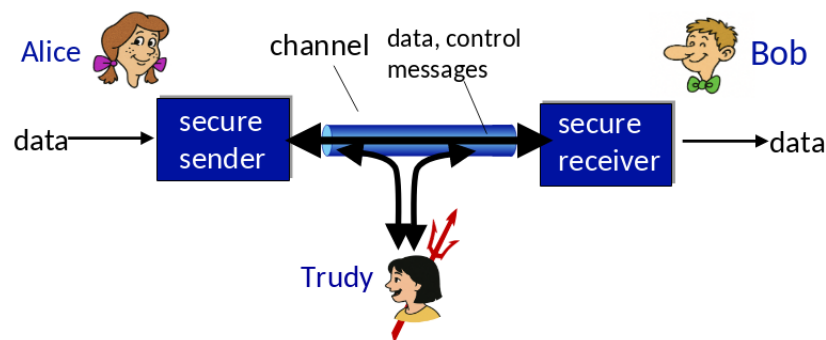
1 Segurança de Redes

1.1 O que é segurança de rede?

- **Confidencialidade:** Apenas o emissor e o receptor pretendido devem “entender” o conteúdo da mensagem.
 - ↪ Emissor: criptografa a mensagem
 - ↪ Receptor: descriptografa a mensagem
- **Autenticação:** O emissor e o receptor desejam confirmar a identidade um do outro.
- **Integridade da mensagem:** O emissor e o receptor desejam garantir que a mensagem não foi alterada (durante a transmissão ou depois) sem detecção.
- **Acesso e disponibilidade:** Os serviços devem ser acessíveis e disponíveis para os usuários.

Exemplo

- Bob e Alice (amantes!) querem se comunicar “com segurança”
- Trudy (intrusa) pode interceptar, apagar e adicionar mensagens



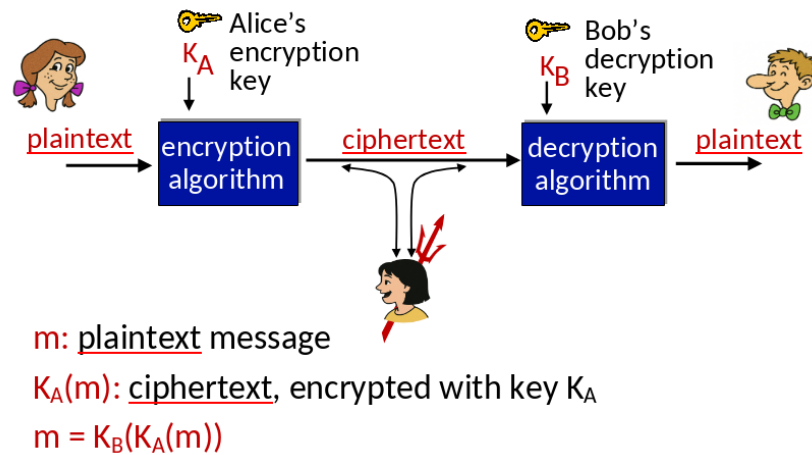
Alice e Bob podem ser:

- Navegador web/servidor para transações eletrônicas (ex: compras online)
- Cliente/servidor de banco online
- Servidores DNS
- Roteadores BGP trocando atualizações de tabelas de roteamento

Trudy, o “bad guy” pode ser:

- **Eavesdrop:** Interceptar mensagens.
- Inserção **ativa** de mensagens em uma conexão.
- **Impersonation:** Pode falsificar (*spoof*) o endereço de origem no pacote (ou qualquer campo do pacote).
- **Hijacking:** “Assumir” uma conexão em andamento removendo o emissor ou o receptor e inserindo-se no lugar.
- **Denial of service:** Impedir que o serviço seja usado por outros (ex: sobrecarregando recursos).

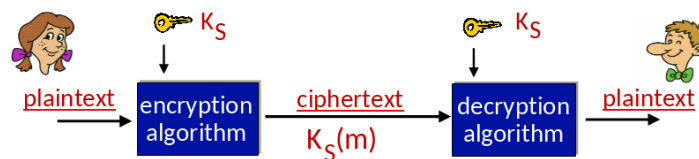
1.2 Princípios da criptografia



Quebrando um esquema de criptografia

- **Ataque somente com texto cifrado (cipher-text only attack):** Trudy possui apenas o texto cifrado (ciphertext) para analisar.
- **Duas abordagens:**
 - ↪ Força bruta: Testar todas as chaves possíveis
 - ↪ Análise estatística
- **Ataque com texto conhecido (known-plaintext attack):** Trudy possui o texto original (plaintext) correspondente ao texto cifrado (ciphertext).
 - ↪ Ex: Em uma cifra monoalfabética, Trudy determina associações para a, l, i, c, e, b, o
- **Ataque com texto escolhido (chosen-plaintext attack):** Trudy pode obter o texto cifrado (ciphertext) a partir de textos que ela mesma escolhe.

Criptografia com chave simétrica



- **Criptografia de chave simétrica:** Bob e Alice compartilham a mesma chave (simétrica): K .
 - ↪ Ex: A chave consiste em conhecer o padrão de substituição em uma cifra de substituição monoalfabética.

Esquema de criptografia simples

- **Cifra de substituições (substitution cipher):** Substitui um elemento por outro.
 - ↪ Cifra monoalfabética: Substitui uma letra por outra.

plaintext: abcdefghijklmnopqrstuvwxyz
 ↓ ↓
ciphertext: mnbvcxzasdfghjklpoiuytrewq

e.g.: Plaintext: bob. i love you. alice
 ciphertext: nkn. s qktc wky. mgsbo

- **Chave de criptografia:** Mapeamento de um conjunto de 26 letras para outro conjunto de 26 letras.

Esquema de criptografia sofisticada

- n cifras de substituição: $M_1, M_2, \dots, M_n$
- Padrão cíclico:
 - ↪ e.g, $n = 4$: $M_1, M_3, M_4, M_3, M_2; M_1, M_3, M_4, M_3, M_2; \dots$
- Para cada novo símbolo do texto original, utiliza-se o próximo padrão de substituição no ciclo.
 - ↪ e.g, dog: d: M_1 ; o: M_3 ; g: M_4 .
- **Chave de criptografia:** n cifras de substituição e o padrão cíclico.
 - ↪ A chave não precisa ser apenas um padrão de n bits.

Criptografia de chave simétrica: DES

- **DES (Data Encryption Standard):**
 - Padrão de criptografia dos EUA [NIST 1993].
 - Chave simétrica de 56 bits, entrada de texto original (plaintext) de 64 bits.
 - Cifra de bloco com encadeamento de blocos (cipher block chaining).
- Quanto seguro é o DES
 - **DES Challenge:** Uma frase criptografada com chave de 56 bits foi descriptografada (por força bruta) em menos de um dia.
 - Não há ataques analíticos eficazes conhecidos.
- Tornando o DES mais seguro:
 - **3DES:** Criptografa 3 vezes com 3 chaves diferentes.

AES: Padrão de Criptografia Avançada (Advanced Encryption Standard)

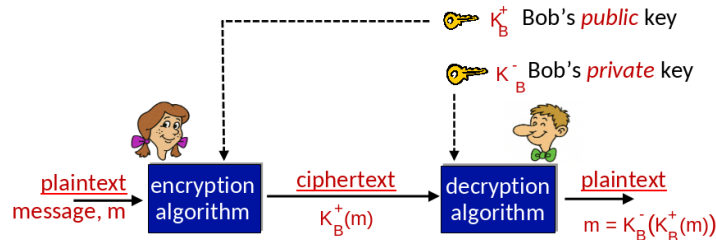
- Padrão de **chave simétrica** do NIST, que substituiu o DES (novembro de 2001)
- Processa dados em blocos de 128 bits.
- Utiliza chaves de 128, 192 ou 256 bits.
- A descryptografia por força bruta (testando cada chave), que leva cerca de 1 segundo no DES, levaria aproximadamente 149 trilhões de anos no AES-128.

Criptografia de Chave Pública (Public Key Cryptography)

- **Criptografia com chave simétrica:**
 - ↪ Exige que o emissor e o receptor conheçam uma chave secreta **compartilhada**.
 - ↪ Como estabelecer essa chave, se nunca se encontraram ?

- **Criptografia de chave pública:**

- ↪ Abordagem radicalmente diferente ([Diffie-Hellman76, RSA78])
- ↪ Emissor e receptor **não compartilham** uma chave secreta.
- ↪ Chave **pública** de criptografia: conhecida por **todos**.
- ↪ Chave **privada** de descriptografia: conhecida apenas pelo **receptor**.



- **Observação:** A criptografia de chave pública revolucionou um campo de 2000 anos (que antes utilizava apenas criptografia simétrica).
 - ↪ Ideias semelhantes surgiram aproximadamente na mesma época, de forma independente nos EUA e no Reino Unido (inicialmente classificadas).

Algoritmos de criptografia de chave pública

- **Algoritmo de criptografia:** É o método ou conjunto de regras matemáticas usado para transformar um texto em forma ilegível.
 - ↪ Exemplos: AES, RSA, DES.
- **Chave de criptografia:** É um valor secreto utilizado pelo algoritmo para realizar a criptografia e a descriptografia.
- **Analogia:**
 - ↪ Algoritmo: tipo de cadeado.
 - ↪ Chave: chave que abre o cadeado.
- **Requisitos:**
 1. É necessário ter $K_b^+(\bullet)$ e $K_b^-(\bullet)$ tal que $K_b^-(K_b^+(m)) = m$.
 2. Dada a chave pública K_b^+ deve ser impossível calcular/computar a chave privada K_b^- .
- **RSA (Rivest-Shamir-Adleman):** O algoritmo de criptografia de chave pública mais amplamente utilizado.

Pré-requisito: Aritmética Modular

- $x \bmod n$ = resto de x quando dividido por n .
- **Propriedades:**
 - $[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$
 - $[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$
 - $[(a \bmod n) \cdot (b \bmod n)] \bmod n = (a \cdot b) \bmod n$
- Assim, $(a \bmod n)^d \bmod n = a^d \bmod n$
- **Exemplo:** $x = 14, n = 10, d = 2$:
 - $(x \bmod n)^d \bmod n = 4^2 \bmod 10 = 16 \bmod 10 = 6$;
 - $x^d = 14^2 = 196$; $x^d \bmod 10 = 6$

RSA: Conceitos Básicos

- **Mensagem:** É apenas um padrão de bits.
- Um padrão de bits pode ser representado de forma **única** por um número inteiro.
- Assim, criptografar uma mensagem é equivalente a criptografar um número.
- Exemplo:
 - ↪ $m = 10010001$. Essa mensagem é representada de forma **única** pelo número decimal 145.
 - ↪ Para criptografar m , criptografamos o número correspondente, o que gera um novo número (o texto cifrado).

RSA: Criação do par de chaves pública/privada

1. Escolher dois números primos grandes p e q (por exemplo, 1024 bits cada).
2. Calcular $n = pq$ e $z = (p - 1)(q - 1)$.
3. Escolher e (com $e < n$) tal que não possua fatores comuns com z (ou seja, e e z são coprimos).
4. Escolher d tal que $ed - 1$ seja exatamente divisível por z (ou seja, $ed \bmod z = 1$).
5. A chave **pública** é $\underbrace{(n, e)}_{K_b^+}$ e a chave **privada** é $\underbrace{(n, d)}_{K_b^-}$.

RSA: Criptografia e Descriptografia

1. Dados (n, e) e (n, d) como calculados anteriormente.
2. Para criptografar uma mensagem m ($m < n$), calcular: $c = m^e \bmod n$.
3. Para descriptografar o padrão de bits recebido c , calcular: $m = c^d \bmod n$.

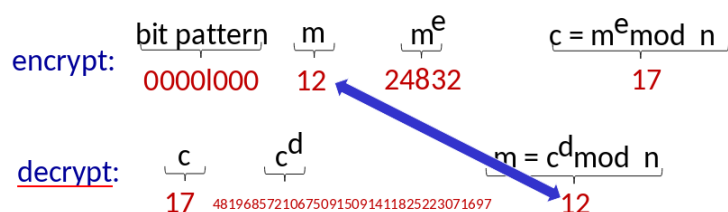
$$m = \left(\underbrace{m^e \bmod n}_c \right)^d \bmod n$$

Exemplo: Bob escolhe: $p = 5, q = 7$. Então, $n = 35$ e $z = 24$.

$e = 5$ (Assim, e e z são relativamente coprimos).

$d = 29$ (Assim, $ed - 1$ é divisível por z , ou seja, $ed \bmod z = 1$).

Criptografando mensagens de 8 bits.



Por que o RSA funciona?

1. Deve-se mostrar que: $c^d \bmod n = m$, onde $c = m^e \bmod n$.
2. Fato: para quaisquer x e y : $x^y \bmod n = x^{(y \bmod n)} \bmod n$
 - ↪ Onde $n = pq$ e $z = (p - 1)(q - 1)$

3. Então,

$$\begin{aligned}
 c^d \bmod n &= (m^e \bmod n)^d \bmod n \\
 &= m^{ed} \bmod n \\
 &= m^{(ed \bmod z)} \bmod n \\
 &= m^1 \bmod n \\
 &= m
 \end{aligned}$$

RSA: Outra Propriedade Importante

$$\underbrace{K_B^-(K_B^+(m))}_{\text{Usa uma chave pública, seguida por uma privada}} = m = \underbrace{K_B^+(K_B^-(m))}_{\text{Usa uma chave privada, seguida por uma pública}}$$

- Isso acontece diretamente da aritmética modular.

$$\begin{aligned}
 (m^e \bmod n)^d \bmod n &= m^{ed} \bmod n \\
 &= m^{de} \bmod n \\
 &= (m^d \bmod n)^e \bmod n
 \end{aligned}$$

Por que o RSA é seguro?

- Se você conhece a chave pública de Bob (n, e) . Quão difícil é determinar d ?
- Essencialmente, é necessário encontrar os fatores de n sem conhecer previamente os dois fatores p e q .
 $\hookrightarrow$ Fato: fatorar um número grande é computacionalmente difícil.

RSA na prática: chaves de sessão

- A exponenciação no RSA é computacionalmente intensiva.
- O DES é pelo menos 100 vezes mais rápido que o RSA.
- Utiliza-se criptografia de chave pública para estabelecer uma **conexão segura** e, em seguida, define-se uma segunda chave — a chave de sessão simétrica — para criptografar os dados;

Chave de Sessão: K_S .

- Bob e Alice usam RSA para trocar uma **chave de sessão** simétrica K_S .
- Após ambos possuírem K_S , utilizam criptografia de chave simétrica.

1.3 Uso da Criptografia de Chave Pública

1.3.1 Autenticação

- **Objetivo:** Bob quer que Alice “prove” a sua identidade para ele.
- **Protocolo ap1.0:** Alice fala “I am Alice”.
- **Problema:** Na internet Bob não consegue “ver”, então Trudy pode simplesmente se declarar como Alice.



Auntificação: Outra Tentativa

- **Objetivo:** Bob quer que Alice “prove” a sua identidade para ele.
- **Protocol ap2.0:** Alice fala “I am Alice” em um pacote IP contendo o seu endereço de IP de origem.
- **Problema:** Trudy pode criar um pacote “spoofing”, falsificado, do endereço IP de Alice.



Auntificação: Uma Terceira Tentativa

- **Objetivo:** Bob quer que Alice “prove” a sua identidade para ele.
- **Protocol ap3.0:** Alice fala “I am Alice” e envia a sua senha secreta para “provar” que é ela mesma.
- **Problema (*Playback Attack*):** Trudy grava o pacote de Alice e, posteriormente, o reproduz para Bob.



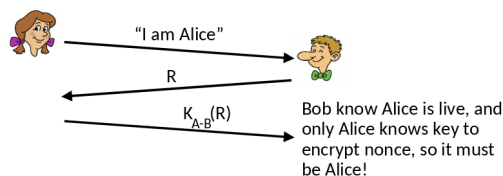
Auntificação: Uma Terceira Tentativa Modificada

- **Objetivo:** Bob quer que Alice “prove” a sua identidade para ele.
- **Protocol ap3.0:** Alice fala “I am Alice” e envia a sua senha secreta criptografada para “provar” que é ela mesma.
- **Problema:** *Playback Attack* continua funcionando.



Auntenticação: Uma Quarta Tentativa

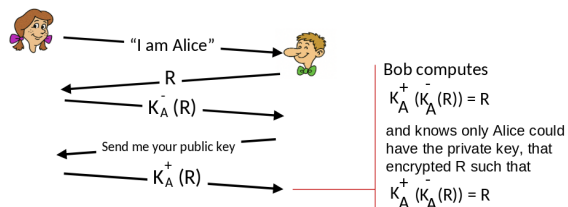
- **Objetivo:** Evitar o *Playback Attack*.
- **nonce:** Um número R usado somente **uma única vez na vida**.
- **Protocol ap4.0:** Para provar que Alice está “viva”, Bob envia Alice nonce, R .
 $\hookrightarrow$ Alice deve retornar R , criptografado com uma chave secreta compartilhada.



Auntenticação: ap5.0

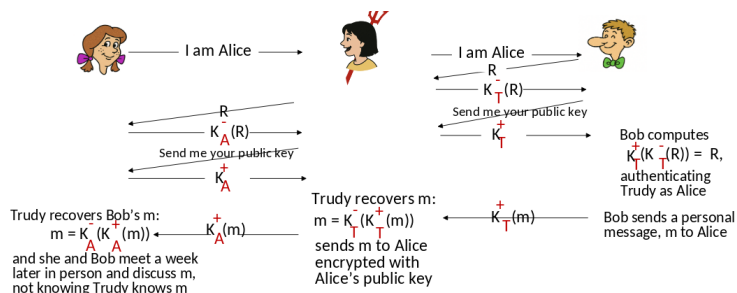
- **ap4.0** requer/exige que chave simétrica seja compartilhada.
- **ap5.0:** Usa o nonce, criptografia de chave pública.

Problema: Como identificar que é realmente Alice conversando com Bob, saber que a chave pública é de fato de Alice.



Auntenticação: ap5.0 - ainda há uma falha!

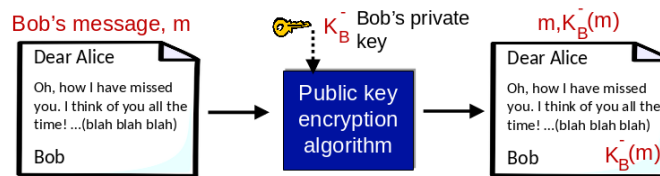
- **Homem (ou Mulher) no meio do ataque:** Trudy se faz passar por Alice (para Bob) e por Bob (para Alice).



1.3.2 Assinaturas Digitais e Integridade de Mensagens

Técnica criptográfica análoga a assinaturas manuscritas:

- O Emissor (Bob) assina digitalmente o documento: ele é o proprietário/criador do documento.
- **Verificável e infalsificável:** O receptor (Alice) pode provar a alguém que Bob, e ninguém mais (incluindo Alice), assinou o documento.
- **Assinatura digital simples para a mensagem m :**
 - $\hookrightarrow$ Bob assina m criptografando-a com sua chave privada K_b , criando uma mensagem “assinada”, $K_b^-(m)$.



- Suponha que Alice recebe a mensagem m com a assinatura: $m, K_b^-(m)$.
- Alice verifica m assinado por Bob aplicando a chave pública K_b^+ de Bob a $K_b^-(m)$, e em seguida, verifica se $K_b(K_b^-(m))^+ = m$.
- Se $K_b(K_b^-(m))^+ = m$, quem assinou m deve ter usado a chave privada de Bob.

Alice verifica, portanto, que:

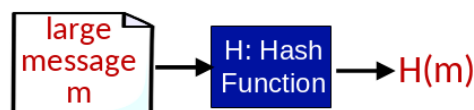
- Bob assinou m .
- Ninguém mais assinou m .
- Bob assinou m não m' .

Não-Repúdio:

- Alice pode levar m e a assinatura $K_b^-(m)$ ao tribunal e provar que Bob assinou m .

Resumos de mensagens (Message Digests) Criptografar mensagens longas com chave pública é computacionalmente caro. **Objetivo:** Obter uma "impressão digital" ("*fingerprint*") digital de comprimento fixo e fácil de calcular.

- Aplica-se uma função *hash* H à mensagem m , obtendo um resumo (*message digests*) de tamanho fixo: $H(m)$.



Propriedades da função *hash*

- Muitos-para-um (many-to-1): várias mensagens diferentes podem gerar o mesmo *hash*.
- Produz um resumo (*message digests*) de tamanho fixo (*fingerprint*), independente-

mente do tamanho da mensagem original.

- Dado um resumo (*message digests*) x , é computacionalmente inviável encontrar uma mensagem m tal que $x = H(m)$.

Checksum da Internet: uma função hash criptográfica fraca O Internet checksum possui algumas propriedades de uma função *hash*:

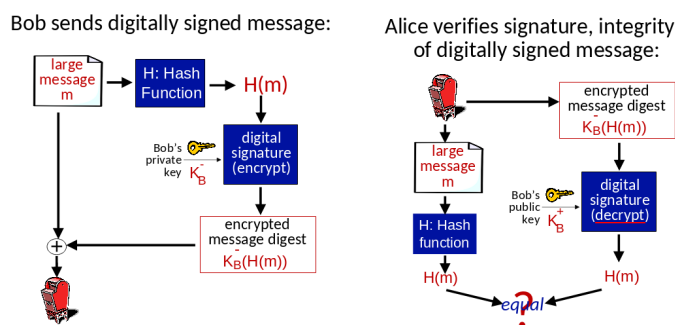
- Produz um resumo de tamanho fixo (soma de 16 bits) da mensagem.
- É muitos-para-um (many-to-one).

No entanto, dado uma mensagem com um determinado valor de hash, é fácil encontrar outra mensagem com o mesmo valor de hash.

message	ASCII format	message	ASCII format
I O U 1	49 4F 55 31	I O U 9	49 4F 55 39
0 0 . 9	30 30 2E 39	0 0 . 1	30 30 2E 31
9 B O B	39 42 D2 42	9 B O B	39 42 D2 42
B2 C1 D2 AC		B2 C1 D2 AC	

different messages
but identical checksums!

Assinatura digital = resumo da mensagem assinada (*message digests*)

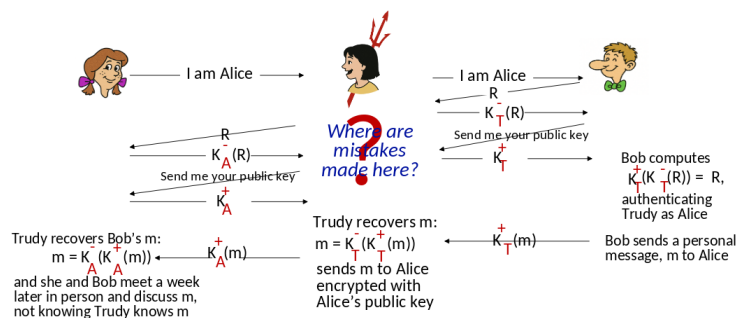


Algoritmos de função *hash*

- A função hash **MD5** é amplamente utilizada (RFC 1321).
 - ↪ Calcula um resumo de mensagem de 128 bits em um processo de 4 etapas.
 - ↪ Dada uma string arbitrária x de 128 bits, parece difícil construir uma mensagem m cujo hash MD5 seja igual a x .
- O **SHA-1** também é utilizado
 - ↪ Padrão dos EUA [NIST, FIPS PUB 180-1].
 - ↪ Produz um resumo de mensagem de 160 bits.

A autenticação: ap5.0 - vamos resolvê-lo !!

- **Relembrando o problema:** Trudy se passa por Alice (para Bob) e por Bob (para Alice).

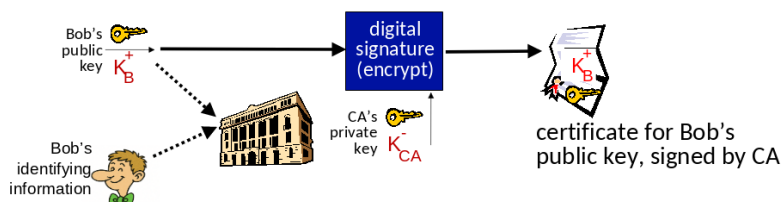


Necessidade de chaves públicas certificadas

- Motivação: Trudy faz uma pegadinha de pizza com Bob.
 - ⇒ Trudy cria um pedido por e-mail: “Prezada Pizzaria, Por favor, entregue para mim quatro pizzas de pepperoni. Obrigado, Bob.”
 - ⇒ Trudy assina o pedido com sua chave privada.
 - ⇒ Trudy envia o pedido para a pizzaria.
 - ⇒ Trudy envia também sua chave pública, mas diz que é a chave pública de Bob.
 - ⇒ A pizzaria verifica a assinatura e então entrega quatro pizzas de pepperoni para Bob.
 - ⇒ Bob nem gosta de pepperoni!

Autoridades Certificadoras de Chaves Públicas (CA)

- **Autoridade certificadora (CA):** Vincula uma chave pública a uma entidade específica, E .
- Entidade (pessoa, site, roteador) registra sua chave pública com o CE e fornece uma “prova de identidade” à CA.
 - ⇒ A CA cria um certificado que associa a identidade E à sua chave pública.
 - ⇒ O certificado, contendo a chave pública de E , é assinado digitalmente pela CA: a CA afirma que “esta é a chave pública de E ”



- Quando Alice quer a chave pública de Bob:
 - ⇒ Obtém o certificado de Bob (diretamente com Bob ou de outra fonte).
 - ⇒ Aplica a chave pública da CA (Autoridade Certificadora) ao certificado de Bob obtendo, assim, a chave pública de Bob.

1.3.3 Acordo de Chave Compartilhada

Usando Criptografia de Chave Pública para Concordar em uma Chave Compartilhada

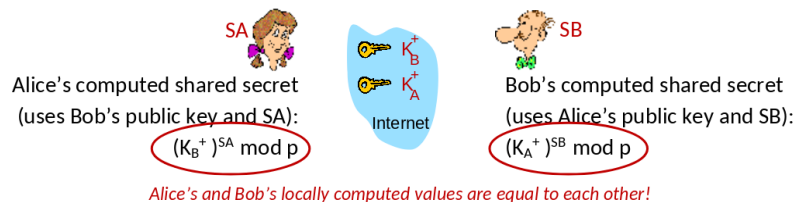
- **Problema:** para Bob e Alice utilizarem técnicas de chave simétrica, eles precisam concordar sobre uma chave compartilhada.
 - ↪ E se eles nunca se encontraram antes?
 - ↪ A criptografia de chave pública também pode resolver esse problema!

Diffie-Hellman (DH) chaves pública/privada: Usa um algoritmo diferente do RSA.

- Bob escolhe valores **públicos**: Um número primo grande p , outro número grande g (com $g < p$).
- Alice e Bob escolhem, independentemente, valores **privados**: SA (de Alice) e SB (de Bob).
- **Chave pública de Alice:** $K_A^+ = g^{SA} \mod p$.
- **Chave pública de Bob:** $K_B^+ = g^{SB} \mod p$.

Diffie-Hellman (DH): derivação da chave compartilhada:

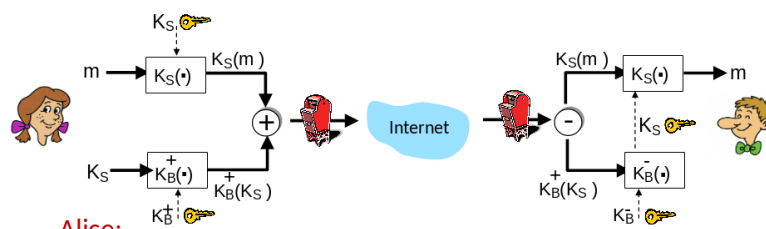
- Alice e Bob calculam um segredo compartilhado em comum sem nunca trocá-lo explicitamente!
- Alice e Bob possuem chaves públicas DH: K_A^+ e K_B^+ , e valores secretos: SA e SB .



1.4 E-mail seguro

Confidencialidade

- Alice quer enviar um e-mail **confidencial**, m , para Bob.



Alice

- Gera uma chave privada **simétrica** aleatória K_S .
- Criptografa a mensagem com K_S (por eficiência).

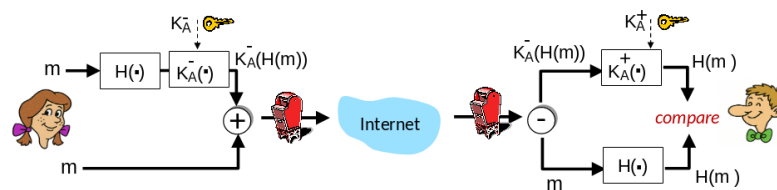
- Também criptografa K_S usando a chave pública de Bob.
- Envia para Bob: $K_S(m)$, mensagem criptografada, $K_B(K_S)^+$, a chave simétrica criptografada com a chave pública de Bob.

Bob

- Usa a sua chave privada para descriptografar e recuperar a chave K_S .
- Usa K_S para descriptografar $K_S(m)$ para a mensagem m .

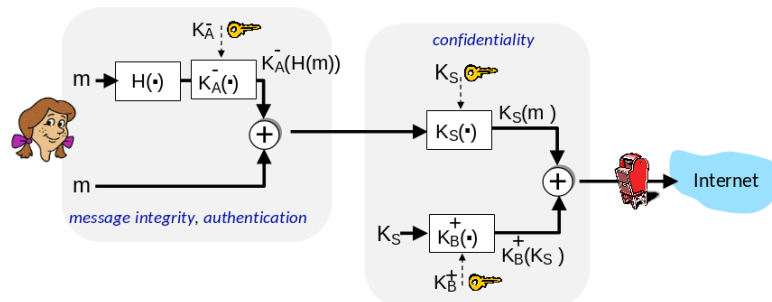
Integridade, Autenticação

Alice quer enviar uma mensagem m para Bob com **integridade da mensagem e autenticação**.



- Alice assina digitalmente o hash da mensagem usando sua chave privada, garantindo integridade e autenticação.
- Envia tanto a mensagem (em claro) quanto a assinatura digital.

Alice envia uma mensagem m para Bob garantindo: **Confidencialidade, Integridade da mensagem e Autenticação**.



- **Alice utiliza três chaves:** Sua chave privada, a chave pública de Bob e uma nova chave simétrica.

1.5 Proteção de conexões TCP: TLS

Segurança na Camada de Transporte - TLS

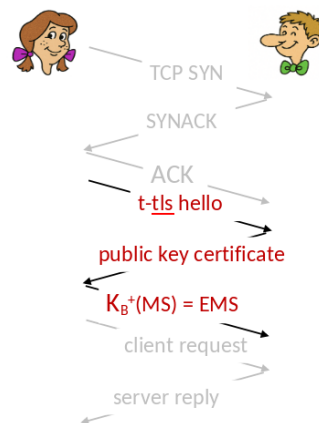
- Protocolo de segurança amplamente utilizado acima da camada de transporte.
 - ↔ Suportado por praticamente todos os navegadores e servidores web: <https> (porta 443).

- Fornece:
 - ↪ **Confidencialidade:** por meio de criptografia simétrica.
 - ↪ **Integridade:** por meio de hashing criptográfico.
 - ↪ **Autenticação:** por meio de criptografia de chave pública.
- Histórico:
 - ↪ Pesquisas iniciais e implementações: programação de rede segura, *secure sockets*.
 - ↪ Secure Sockets Layer (SSL) descontinuado [2015].
 - ↪ TLS 1.3 [2018].

O que é Necessário ?

- Vamos construir um protocolo TLS simplificado (*toy TLS*), chamado **t-TLS**, para entender o que é necessário!
- Algumas peças já foram vistas:
 - ↪ **Handshake:** Alice e Bob usam seus certificados e chaves privadas para se autenticarem mutuamente e trocar ou criar um segredo compartilhado.
 - ↪ **Derivação de chaves:** Alice e Bob usam o segredo compartilhado para derivar um conjunto de chaves.
 - ↪ **Transferência de dados:** Transferência contínua (stream) de dados: os dados são enviados como uma série de registros.
 - Não se trata apenas de transações únicas.
- **Encerramento da conexão:** Mensagens especiais são usadas para encerrar a conexão de forma segura.

t-tls: Handshake inicial



- Bob estabelece uma conexão TCP com Alice.
- Bob verifica que Alice é realmente Alice.
- Bob envia para Alice uma chave secreta mestra (MS), usada para gerar todas as outras chaves da sessão TLS.
- Problemas potenciais:
 - ↪ São necessários 3 RTT (*round-trip times*) antes que o cliente possa começar a receber dados (incluindo o handshake do TCP).

t-tls: Chaves criptografadas

- É considerado uma má prática usar a mesma chave para mais de uma função criptográfica.
 - ↪ Devem ser usadas chaves diferentes para o **MAC** (*Message Authentication Code*) e Criptografia.
- Quatro chaves:
 - ↪ K_c : Chave de criptografia para dados enviados do cliente para o servidor.
 - ↪ M_c : Chave de **MAC** para dados enviados do cliente para o servidor.
 - ↪ K_s : Chave de criptografia para dados enviados do servidor para o cliente.
 - ↪ M_s : Chave de **MAC** para dados enviados do servidor para o cliente.
- As chaves são derivadas a partir de uma **Função de derivação de chaves** (*KDF* — *Key Derivation Function*).
 - ↪ Usa a chave mestra (MS) e possivelmente pode usar também dados aleatórios adicionais para gerar novas chaves para a sessão.

t-tls: Dados criptografados

- Lembrete: o TCP fornece uma **abstração de fluxo contínuo de bytes**.
- Pergunta: Podemos criptografar os dados diretamente no fluxo (stream) ao escrevê-los no socket TCP?
- Resposta: Onde colocar o MAC? Se for no final, não há integridade da mensagem até que todos os dados sejam recebidos e a conexão seja encerrada!!
- **Solução**: Dividir o fluxo em uma série de “registros” (*records*).
 - ↪ Cada registro do cliente-servidor carrega um **MAC**, gerado com M_c .
 - ↪ O receptor pode processar cada registro conforme ele chega.
- Cada registro do t-tls é criptografado usando uma chave simétrica K_c enviado pelo TCP.



- Possíveis ataques ao fluxo de dados ?
 - ↪ **Reordenação**: Um atacante (*man-in-the-middle*) intercepta segmentos TCP e altera a ordem (manipulando números de sequência no cabeçalho TCP, que não é criptografado).
 - ↪ **Repetição - replay**.
- Soluções:
 - ↪ Usar números de sequência do TLS (dados + número de sequência TLS incluídos no MAC).
 - ↪ Usar nonce, valor único/aleatório.

t-tls: Encerramento de conexão

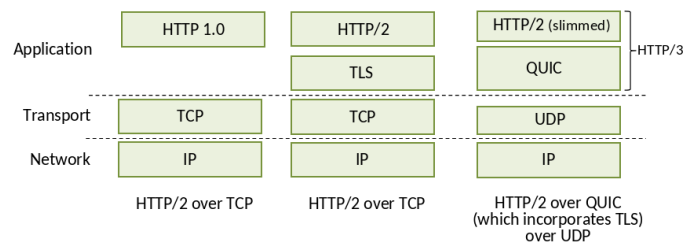
- Ataque de Truncamento:
 - ↪ Um atacante forja um segmento TCP de encerramento.
 - ↪ Um ou ambos os lados acreditam que há menos dados do que realmente foram

enviados.

- **Solução:** Usar tipos de registros, com um tipo para o fechamento.
 $\hookrightarrow 0 \rightarrow \text{Dados}, 1 \rightarrow \text{Fechamento}.$
- MAC agora computa usando os dados, tipo e número de sequência.

$$K_c(\text{length} \mid \text{type} \mid \text{data} \mid \text{MAC})$$

- TLS fornece uma API que qualquer aplicação pode usar.
- Uma visão HTTP do TLS:

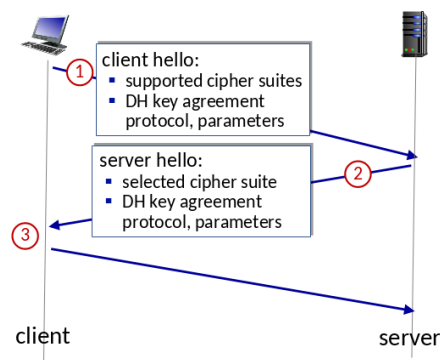


TLS 1.3: Conjunto de cifras (*cipher suite*)

- “Cipher suite”: Algoritmos que podem ser usados para geração de chaves, criptografia, MAC e assinatura digital.
- TLS 1.3: Possui um conjunto de cifras mais limitado do que o TLS 1.2 (2008).
 - $\hookrightarrow$ Apenas 5 opções, em vez de 37.
 - $\hookrightarrow$ Exige o uso de Diffie-Hellman (DH) para troca de chaves, antes podia ser DH ou RSA.
 - $\hookrightarrow$ Utiliza “criptografia autenticada” (*authenticated encryption*), combina algoritmos de criptografia em autenticação em um único processo, antes fazia isso em etapas separadas.
 - $\hookrightarrow$ O HMAC utiliza a função hash criptográfica: SHA-256 ou SHA-384.

TLS 1.3 handshake: 1 RTT

1. client TLS hello msg:
 - Adivinha o protocolo de acordo de chaves e os parâmetros.
 - Indica os conjuntos de cifras suportados.
2. server TLS hello msg escolhe:
 - Protocolo de acordo de chaves, parâmetros.
 - Conjunto de cifras.
 - Certificado assinado pelo servidor.
3. client
 - Verifica o certificado do server.
 - Gera uma chave.
 - Pode agora fazer requisições na aplicação (HTTPS GET).



TLS 1.3 handshake: 0 RTT

- Mensagem inicial de hello contém dados criptografados pela aplicação.
 - ↔ “Retomando” a conexão anterior entre o cliente e o servidor.
 - ↔ Dados da aplicação criptografados usando o “segredo mestre de retomada” da conexão anterior.
- Vulnerável a ataques **replay**!!
 - ↔ Talvez seja aceitável para requisições HTTP GET ou requisições do cliente que não modificam o estado do servidor.

